

PROJECT SYNOPSIS

CI/CD Pipeline for Java Application using Docker and Maven

using Docker, Maven, Java, Git & GitHub

Student Name
Sundram Singh

Registration Number
12312641

1. Problem Statement

In modern software development, applications are frequently updated with new features and bug fixes. Manual processes for building, testing, and deploying applications are time-consuming, error-prone, and inefficient.

There is a need for an automated system that can:

- Build the application automatically
- Test and package it reliably
- Deploy it without manual intervention

This project addresses these challenges by implementing a CI/CD pipeline that automates the entire workflow from code commit to deployment, ensuring faster and more reliable software delivery.

2. Tools and Technologies Used

Tool / Technology	Purpose
Docker	Containerization of application versions
java	Application development
Apache Maven	Build automation and dependency management
Git & GitHub	Version control and project hosting
GitHub Actions	CI/CD pipeline automation

3. System Architecture

The pipeline follows an automated sequence from code push to deployment:

Developer → GitHub Repository →
GitHub Actions →
Maven Build & Test → Docker
Image Creation →
Run Container → Application
Deployment

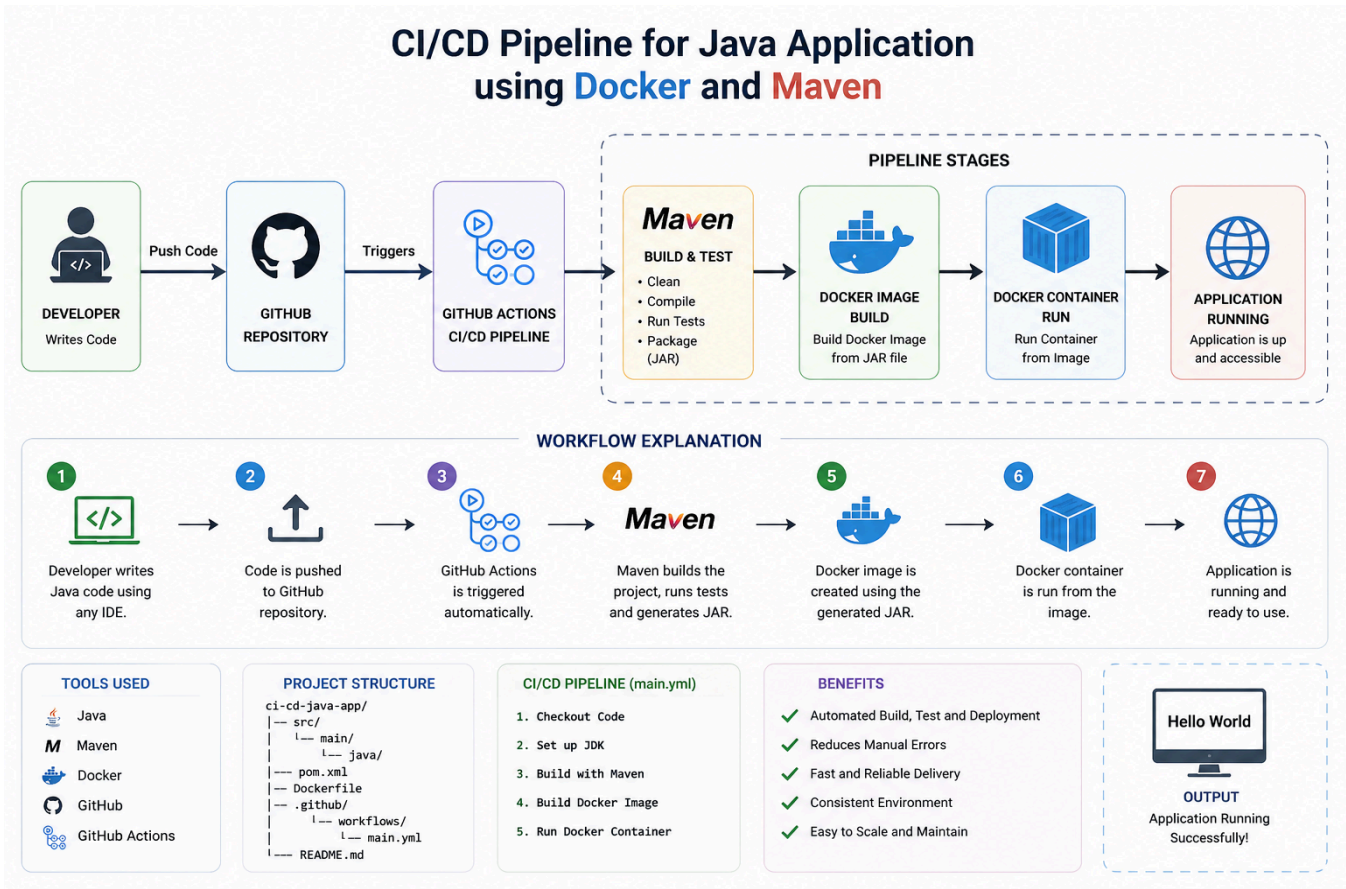
Project Synopsis | CI/CD Pipeline for Java Application using Docker and Maven

- Developer pushes code to GitHub
- GitHub Actions triggers the pipeline
- Maven builds and packages the application
- Docker creates a container image
- Application runs inside a container

Traffic is switched dynamically without stopping the application.

4. Workflow Explanation

Step 1	The developer writes Java code and pushes it to the GitHub repository.
Step 2	GitHub Actions automatically triggers the CI/CD pipeline on each push
Step 3	Maven compiles the code, runs tests, and generates a . jar file. .
Step 4	A Docker image is created using the generated . jar file.
Step 5	The Docker container is built and executed.
Step 6	The application runs successfully inside the container and becomes accessible.



5. Expected Results

- Automated build and deployment process
- Successful packaging of Java application using Maven
- Containerized deployment using Docker
- Reduced manual effort and human errors
- Faster and reliable application delivery

6. Conclusion

This project demonstrates the implementation of a basic **CI/CD pipeline** for a Java application using Maven, Docker, and GitHub Actions.

By automating the build and deployment process, the system improves efficiency, consistency, and reliability. It reflects key DevOps practices such as continuous integration, automation, and containerization.

The project provides a strong foundation for understanding real-world DevOps workflows and can be extended further for advanced deployment strategies.

Why This Project Stands Out

- Demonstrates real CI/CD pipeline implementation
- Uses industry-standard tools (Maven, Docker, GitHub Actions)
- Fully automated workflow from code to deployment
- Beginner-friendly yet conceptually strong
- Easy to explain in viva and practical demonstrations

7. Future Scope

- Deploy the application on **cloud platforms (AWS/Azure)** for real-world usage
- Integrate **automated testing** to improve code quality and reliability
- Implement **Docker image versioning and rollback mechanisms**
- Extend to **advanced deployment strategies** like Blue-Green or Kubernetes